

Flappy Continents

Flappy Continents (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Games · **Assessed:** 2026-07-28 · **Status:** awaiting instructor approval

Proposal (as submitted): A Flappy Bird–style one-touch game where the characters and environments are themed to the 7 continents, with flying (moving) platforms in the sky in addition to static grounded platforms.

Adjustments required for approval:

- The obstacle system must be a genuine **data-driven procedural pattern engine** — flying/moving platforms with authored movement patterns (path, speed, phase), gap and spacing curves — not a handful of hard-coded static pipes. This is what makes the real-time C++ loop load-bearing enough for six CS students.
- The **7 continents must be a data-driven content pipeline** (each continent = biome config + parallax layers + hazard/platform set + character skins, loaded from data with hot-reload), not seven hard-coded scenes.
- Ship a **custom pattern & continent-theme editor** as the project's tool (M2 G2): author obstacle/platform patterns and continent themes and preview them live, without recompiling.
- **Leaderboards must run through Firebase with server-checkable score integrity** — each run reproducible from a seed + input log (the Replay System requirement) so submitted scores are verifiable, not trusted client values.

References:

- [Flappy Bird \(official\)](#)
- [Jetpack Joyride Classic — Halfbrick \(official\)](#)
- [Subway Surfers — "World Tour" themed locations \(Wikipedia\)](#)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Flappy Continents
External Database (Firebase)	✓
Authentication »	»
Analytics »	»
C++ Performant Service	●
Real-time C++ Simulation Loop	✓
Application Communication	●
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Local Backend	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	✕
Replay System — record & playback of game sessions (new)	✓

How each requirement applies:

Platform fit: **Web browser on mobile and/or Native mobile (Pixel 8a/9a)** — one-touch tap play is mobile-native; PC is the development/debug vehicle only.

Front end: Parallax continent rendering, character + skin sprites, moving/flying platforms, tap input, score HUD, menus and continent select.

Languages: C++, GLSL.

Back end: Fixed-timestep physics, the procedural pattern/spawn engine, scoring & run statistics, continent/theme & pattern data, Firebase leaderboards and unlock persistence. **Languages:** C++.

Backend scope: Local now, server-ready (T2) — the score-validation service can lift onto the Linux server in T2.

- *Simulation loop:* a fixed-timestep loop runs the tap-to-flap physics and ticks the moving/flying-platform pattern spawner every frame — deterministic and load-bearing, not decorative.
- *Data-driven:* continents (biome, parallax layers, hazard/platform sets), obstacle/platform movement patterns, character skins and per-continent difficulty curves are all defined as JSON with hot-reload.
- *Custom tool:* a pattern & continent-theme editor — place and tune platform patterns, assemble a continent's layers and hazards, preview live.
- *External DB:* Firebase leaderboards + unlocked-continent/skin persistence. *Auth »:* optional accounts for cross-device leaderboards (T2). *Analytics »:* per-continent difficulty/drop-off funnels (T2).
- *2D:* parallax continent backdrops, sprite characters, animated flying platforms, flap/score/crash VFX. *3D:* not needed. *Networking:* none — leaderboards are asynchronous through Firebase, not a custom real-time layer.
- *Replay System:* each run is recorded as seed + timestamped input log and re-simulated deterministically — it validates leaderboard scores and powers a share-a-run feature.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.md` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech:** SDL2 shell with a fixed-timestep loop driving tap-to-flap gravity/impulse and the scrolling world; low-latency single-touch input; C++17/20, CMake, Git.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1 G5 physics and M1 G7 pattern spawner run on this tick; M2 G12 replays it.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Continents, parallax layers, platform-movement patterns, hazard sets, character skins and difficulty curves as JSON data assets (nlohmann/json) with hot-reload.
- **Prerequisites:** G1 (loop & entity update hooks).
- **Feeds into:** M1 G4/G7 (they consume this data); M2 G2 (the editor authors this format).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (frame time, active platforms, collision debug), Tracy profiling, Catch2 tests; stress test = an auto-flap bot running dense patterns for a simulated long session; Windows + Linux CI.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2 G3; the deterministic behaviour M2 G12 relies on.

- **G5 Physics & Collision** — flap gravity/impulse and collision against gaps, grounded platforms and moving/flying platforms.

Tech & dependencies

- **Tech:** Custom 2D integrator for the bird plus AABB/swept collision against static and moving platforms (GLM); deterministic so replays and server validation match.
- **Prerequisites:** G1 (fixed tick), G2 (platform definitions).
- **Feeds into:** The core fail condition; M2 G12 (replays re-run this sim).
- **G7 Procedural Level Generation** — the moving/flying-platform pattern spawner that makes the endless run interesting.

Tech & dependencies

- **Tech:** Seeded pattern spawner composing authored platform patterns (path, speed, phase) into an endless stream with difficulty-curve pacing per continent; identical seed → identical run.
- **Prerequisites:** G1 (tick), G2 (patterns as data).
- **Feeds into:** M2 G2 (editor authors patterns), M2 G12 (seed drives replays).
- **G4 2D Graphics** — parallax continent rendering and sprite/animation drawing.

Tech & dependencies

- **Tech:** Layered parallax scrolling per continent, sprite batching for character/skins and platforms, FreeType score text; SDL2/OpenGL, GLSL, stb_image.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** Every continent's look; M2 G4 (advanced VFX) builds on it.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech:** Tracy-guided optimization of spawn/render hot paths, sprite-atlas memory budgeting, and Emscripten/WASM + Pixel 8a/9a validation of the tap loop.
- **Prerequisites:** All M1 goals.
- **Feeds into:** The mobile deliverable (M2 G15) and showcase stability. AC includes: showcase build runs on the mobile target.
- **G2 ★ Content Editor — Core** — the pattern & continent-theme editor (the project's primary tool).

Tech & dependencies

- **Tech:** Place/tune platform patterns on a timeline, assemble a continent (parallax layers, hazards, skins), JSON round-trip with live in-editor preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/OpenGL renderer (pattern/continent preview).
- **Prerequisites:** M1 G2 (data format), M1 G7 (spawner used for preview).
- **Feeds into:** All 7 continents' content; the content-creation showcase.

• G3 ★ CI — CMake, Code Quality, Build Standards & Packaging

Tech & dependencies

- **Tech:** CMake presets, clang-format/clang-tidy gates, packaging of native + Emscripten web builds via GitHub Actions.
- **Prerequisites:** M1 G3.
- **Feeds into:** Release/showcase builds of every goal.

• G8 Firebase — Accounts, Leaderboards & Analytics — global and per-continent leaderboards.

Tech & dependencies

- **Tech:** Firestore leaderboards + unlocked-continent/skin persistence, optional Auth accounts, Analytics events for per-continent difficulty — REST from C++.
- **Prerequisites:** M1 G2 (score/run data model).
- **Feeds into:** Competitive showcase; validated by M2 G12.

• G12 Replay System — Record & Playback — deterministic replays for score validation and share-a-run.

Tech & dependencies

- **Tech:** Record seed + timestamped input log; re-simulate through the deterministic tick; playback controls; server-side re-run validates submitted scores.
- **Prerequisites:** M1 G1 (deterministic loop), M1 G5 (deterministic physics), M2 G8 (scores to validate).
- **Feeds into:** Leaderboard integrity and the share-a-run showcase feature.

• G15 Mobile Platform Port — the first-class mobile output.

Tech & dependencies

- **Tech:** Touch-tap mapping, safe-area/thumb-zone layout, on-device frame budgeting; Android NDK or Emscripten build for the reference phone.
- **Prerequisites:** M2 G1 (optimization & platform validation).
- **Feeds into:** The native-mobile deliverable named in the platform fit.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — seven themed continents of environments, character skins and parallax layers is the dominant art surface; a solo designer carries the single-mechanic loop, feel and progression (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)).

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents. Design goals for Games come from the benchmark-anchored **Games Design pool** (`design-goals-pool-games.pdf`).

- **Design M1:**

- D1 ★ Features Design Document (benchmark-informed — Flappy Bird core loop + Subway Surfers "World Tour" theming as primary/secondary benchmarks; scope calibrated to a ★ rating)
- D2 Core Gameplay Loop Prototype (tap-to-flap toy proving the feel — default mandated-designer pick)
- D4 Game Feel & Touch Interaction Specification (tap response, forgiveness, moving-platform readability — only with a 2nd designer)

- **Design M2:**

- D1 ★ Features Design Iteration & Showcase Readiness (the 7-continent scope proven demoable, sharpened against the benchmark)
- D8 Monetization & Scope Integrity (cosmetic continent/character skins, clean model; cut anything that endangers a demoable 7-continent showcase — only with a 2nd designer)

- **UI/UX M1** (graded by the Design instructor for the Games category):

- U1 ★ UI Standards & Wireframe Baseline (main menu, continent select, tap-readable score HUD)

- **UI/UX M2:**

- U1 ★ Functional UI & Usability (Games required screens: Start, Pause, Resume, Quit-with-confirmation, How to Play, continent select, readable HUD; first-time users unaided)

- U2 ★ UX Validation & Final Polished UI (tested with real players; animated, alive)
- U5 Mobile UX Adaptation & Ergonomics (thumb-zone tap, one-handed play on Pixel 8a/9a)
- **Art M1:** 2 artists (character + 7-continent environment surface)
 - A1 ★ (mockups: main menu + one continent scene)
 - A2 Character Concepts (base character + per-continent skins)
 - A3 Environment, Props & VFX Concepts (continent biomes, parallax layers, flying-platform art)
- **Art M2:**
 - A1 ★ Critical asset production across the shipped continents
 - A3 Environment Alive (parallax ambient animation per continent)
 - A5 VFX (flap trails, score bursts, crash)
- **TEAM M1:**
 - TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (flap, score, crash; one continent's ambience), suitable and correctly triggered.
 - TA2 ★ Audio Engine Functionality — data-driven audio pipeline (swap a continent's music/SFX without recompiling) + reliable multi-channel playback.
- **TEAM M2:**
 - TA1 ★ Overall Audio Experience — Games floor: ≥1 BGM per continent theme + ≥6 SFX types in the showcase build, correctly triggered.
 - TA2 ★ Audio Engine Functionality — audio de/serialised, continent-music crossfades, full multi-channel playback the build needs.
 - TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the pattern/theme editor), prototype demo, team post-mortem (wk14).